

Interrupts with Microblaze

By the end of the lab you will be able modify the hardware platform to support interrupts and configure them in software.

Adding Interrupts

Microblaze has a dedicated interrupt interface port consisting of several signals. The most important signal is called `interrupt`, and a transition on this pin will issue an interrupt to the processor, causing it to branch to the interrupt vector.



Figure 1: Microblaze interrupt interface and pseudocode for interrupt configuration

Modify the previous hardware platform, such that the timer can issue interrupts to Microblaze. After regenerating the bitstream, create a new vitis project and configure the timer peripheral *and* microblaze to respectively generate and respond to an interrupt. Validate your implementation ensuring that the Interrupt Service Routine (ISR) is invoked when an interrupt is generated.

Homework – Interrupt driven PWM LED controller

The homework's goal is to improve the previously designed software based PWM by moving it to the background using interrupts. The main code should only monitor the user push buttons. Potential steps:

1. Configure the timer to generate a periodic interrupt, to serve as timebase.
2. Move the software based PWM sequence to the ISR while monitoring for user input (button press) in the main loop. Do you observe any difference ?

Optional: Supporting only one interrupt source is limiting. Adding an interrupt controller enables prioritized handling of multiple sources, allowing all I/O tasks to run in the background and fully freeing the main loop.